



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

IoT Challenge 2 - Packet Sniffing

TESI DI LAUREA MAGISTRALE IN
TELECOMMUNICATION ENGINEERING

Author: **Hong CHEN** [REDACTED]
Mengzhen LI [REDACTED]

Student ID: [REDACTED] & [REDACTED]

Advisor: Prof. Name Surname

Co-advisors:

Academic Year: 2025-04

Contents

Contents	i
1 PART 1: Questions on the PCAP file	1
1.1 CQ1: How many different Confirmable PUT requests obtained an unsuccessful response from the local CoAP server?	1
1.2 CQ2: How many CoAP resources in the coap.me public server received the same number of unique Confirmable and Non Confirmable GET requests?	3
1.3 CQ3: How many different MQTT clients subscribe to the public broker HiveMQ using multi-level wildcards?	4
1.4 CQ4: How many different MQTT clients specify a Last Will Message to be directed to a topic having as first level "university"?	5
1.5 CQ5: How many MQTT subscribers receive a last will message derived from a subscription without a wildcard?	6
1.6 CQ6: How many MQTT publish messages directed to the public broker mosquitto are sent with the retain option and use QoS "At most once"? . .	7
1.7 CQ7: How many MQTT-SN messages on port 1885 are sent by the clients to a broker in the local machine?	8

1 | PART 1: Questions on the PCAP file

1.1. CQ1: How many different Confirmable PUT requests obtained an unsuccessful response from the local CoAP server?

The code field in CoAP is an 8-bit unsigned integer (uint8_t) used to represent request methods or response status codes. Format:

8-bit CoAP Code: C C C M M M M M

First 3 bits: Represents Class

- 0 = Requests (GET, POST, PUT, DELETE)
- 2 = Success Response (2.xx)
- 4 = Client Error (4.xx)
- 5 = Server Error (5.xx)

Last 5 bits: Represents Specific Code Details

coap.code(HEX)	coap.code(BIN)	Meaning
0x01	000 00001	GET
0x02	000 00010	POST
0x03	000 00011	PUT
0x04	000 00100	DELETE

Table 1.1: Request Method COdes

The filtering was performed using Tshark with the following criteria:

coap.code(DEC)	coap.code(HEX)	coap.code(BIN)	Meaning
128	0x80	100 00000	4.00 Bad Request
...	...	...	...
132	0x85	100 00100	4.04 Not Found
133	0x85	100 00101	4.05 Method Not Allowed
...	...	...	...
160	0xA0	101 00000	5.00 Internal Server Error
...	...	...	...

Table 1.2: Client Errors(4.xx) and Server Errors (5.xx)

Only CoAP packets were considered. Only packets involving localhost (IPv4 127.0.0.1 or IPv6 ::1) were included.

```
ip.src == 127.0.0.1 || ip.dst == 127.0.0.1 || ipv6.src == ::1 ||
  ↪ ipv6.dst == ::1
```

Only Confirmable (CON) PUT requests were selected.

```
coap.type == 0 && coap.code == 3
```

The response must have a CoAP response code (128).

```
coap && coap.mid == <MID> && coap.code >= 128
```

Result Analysis:

For **CQ1** ("How many different confirmable PUT requests obtained an unsuccessful response from the local CoAP server?"), we identified **22** distinct Confirmable PUT requests that received an unsuccessful response from the local CoAP server. These requests were filtered according to the type of CoAP message and the response code and the results were extracted accordingly.

Listing 1.1: CQ1

```
#!/bin/bash

input_pcap="challenge2.pcapng"
output_csv="result.csv"

echo "Request Type,Request Code,Request MID,Source IP,Destination
  ↪ IP,Response Type,Response Code" > "$output_csv"

tshark -r "$input_pcap" -T fields \
  -e coap.type \
  -e coap.code \
  -e coap.mid \
  -e ip.src \
  -e ip.dst \
```

```

-Y 'coap && (ip.src == 127.0.0.1 || ip.dst == 127.0.0.1 || ipv6
↪.src == ::1 || ipv6.dst == ::1)' | \
while IFS=$'\t' read -r type code mid src dst; do
  if [[ "$type" == "0" && "$code" == "3" ]]; then # Filter PUT
    ↪ requests
    response=$(tshark -r "$input_pcap" -T fields \
      -e coap.type \
      -e coap.code \
      -e coap.mid \
      -e ip.src \
      -e ip.dst \
      -Y "coap && coap.mid == $mid && coap.code >= 128 && (ip
        ↪.src == 127.0.0.1 || ip.dst == 127.0.0.1 || ipv6.
        ↪src == ::1 || ipv6.dst == ::1)")

    if [[ -n "$response" ]]; then
      response_type=$(echo "$response" | cut -f1)
      response_code=$(echo "$response" | cut -f2)
      response_src=$(echo "$response" | cut -f3)
      response_dst=$(echo "$response" | cut -f4)

      echo "$type,$code,$mid,$src,$dst,$response_type,
        ↪$response_code" >> "$output_csv"
    fi
  fi
done

echo "Processing completed. Results saved to $output_csv"
}

```

1.2. CQ2: How many CoAP resources in the coap.me public server received the same number of unique Confirmable and Non Confirmable GET requests?

The question requires us to compare the number of unique Confirmable and Non-Confirmable GET requests received by each CoAP resource on the coap.me public server.

Confirmable GET requests (CON GET): Messages where:

```
coap.type == 0 && coap.code == 1 && ip.dst == 134.102.218.18
```

Non-Confirmable GET requests (NON GET): Messages where:

```
coap.type == 1 && coap.code == 1 && ip.dst == 134.102.218.18
```

The goal is to count how many resources (identified by their URI) received the same number of unique Confirmable and Non-Confirmable GET requests, ensuring that this number is greater than zero.

Filter	Packet	Type	Length	Info
ip.dst == 134.102.218.18 && coap.type == 0 && coap.code == 1	60	CON	14	MID:44935, GET, TKN:1d ba f2 b5 33 a1 25 05, Block #1, /3
	58	CON	19	MID:23556, GET, TKN:cd 19 ee 82 79 f5 97 2e, /3
	60	CON	19	MID:23557, GET, TKN:cd 19 ee 82 79 f5 97 2e, Block #1, /3
	62	CON	45	MID:49271, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, /large
	64	CON	45	MID:49272, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #1, /large
	64	CON	45	MID:49273, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #2, /large
	64	CON	45	MID:49274, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #3, /large
	64	CON	45	MID:49275, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #4, /large
	64	CON	45	MID:49276, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #5, /large
	64	CON	45	MID:49277, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #6, /large
	64	CON	45	MID:49278, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #7, /large
	64	CON	45	MID:49279, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #8, /large
	64	CON	45	MID:49280, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #9, /large
	64	CON	45	MID:49281, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #10, /large
	64	CON	45	MID:49282, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #11, /large
	64	CON	45	MID:49283, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #12, /large
	64	CON	45	MID:49284, GET, TKN:d8 b3 4c 4c 45 1e 53 3f, Block #13, /large
	65	CON	14	MID:56553, GET, TKN:f0 3c b9 1d 7b c3 b6 a6, /validate
	63	CON	14	MID:38178, GET, TKN:a8 dd ec fa 8c 6d d0 fb, /secret
	72	CON	14	MID:6332, GET, TKN:92 d6 1f 34 ba 39 1d 9b, /location-query
	66	CON	14	MID:64172, GET, TKN:8d 34 ca 9f 96 2e 4b 8f, /location1
	62	CON	14	MID:61114, GET, TKN:c4 91 70 66 bb 68 09 c6, /hello
	58	CON	14	MID:63075, GET, TKN:c1 31 51 b2 d9 91 ea 58, /3
	60	CON	14	MID:63076, GET, TKN:c1 31 51 b2 d9 91 ea 58, Block #1, /3
ip.dst == 134.102.218.18 && coap.type == 1 && coap.code == 1	61	NON	15	MID:42045, GET, TKN:09 9f 15 10 72 0b 32 27, /sink
	65	NON	15	MID:48681, GET, TKN:b1 83 ff cd 5c 5a fe 5e, /weird333
	64	NON	15	MID:51115, GET, TKN:c1 15 4c 25 80 7b e6 ca, /weird33
	62	NON	45	MID:53322, GET, TKN:ff 2e e2 9a 0a d1 2d be, /large
	64	NON	45	MID:53323, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #1, /large
	64	NON	45	MID:53324, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #2, /large
	64	NON	45	MID:53325, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #3, /large
	64	NON	45	MID:53326, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #4, /large
	64	NON	45	MID:53327, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #5, /large
	64	NON	45	MID:53328, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #6, /large
	64	NON	45	MID:53329, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #7, /large
	64	NON	45	MID:53330, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #8, /large
	64	NON	45	MID:53331, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #9, /large
	64	NON	45	MID:53332, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #10, /large
	64	NON	45	MID:53333, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #11, /large
	64	NON	45	MID:53334, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #12, /large
	64	NON	45	MID:53335, GET, TKN:ff 2e e2 9a 0a d1 2d be, Block #13, /large
	62	NON	14	MID:55055, GET, TKN:cc 7d da 66 1d cc 64 f2, /weird
	64	NON	14	MID:57052, GET, TKN:5e d8 f0 40 d5 c7 bb e4, /weird44
	61	NON	14	MID:62249, GET, TKN:d6 23 4e 30 74 39 19 4a, /sink
	58	NON	14	MID:63323, GET, TKN:db 73 87 93 f8 cd 51 01, /4
	65	NON	14	MID:63723, GET, TKN:b1 10 a1 2d 77 33 a0 bf, /separate
	65	NON	14	MID:63767, GET, TKN:cd 03 3c 58 6a 63 84 22, /validate

Figure 1.1: CQ2

After extracting and analyzing the data, the number of CoAP resources that received the same number of Confirmable and Non-Confirmable GET requests is :

Resource	Confirmable GET Count	Non Confirmable GET Count
large	14	14
secret	1	1
validate	1	1

Table 1.3: CQ2: CoAP GET ReqComp

Concluding analysis: We found three resources `/large`, `/secret`, and `/validate` that each received the same number of Confirmable (CON) and Non-confirmable (NON) GET requests, with both counts greater than zero. **Final Answer: 3**

1.3. CQ3: How many different MQTT clients subscribe to the public broker HiveMQ using multi-level wildcards?

The task requires identifying how many different MQTT clients subscribe to the public broker HiveMQ using multi-level wildcards.

In MQTT, wildcards are used in subscription topics to match multiple topic levels. There are two types of wildcard:

- **Single-Level Wildcard (+):** Matches one topic level.
- **Multi-Level Wildcard (#):** Matches multiple topic levels, starting from the current level onward. It must be used at the end of a topic string.

(mqtt.msgtype == 8 && mqtt.topic contains "#") && (ip.dst == 18.192.151.104)						
No.	Time	Source	Destination	Protocol	Length	Info
3293	20.163021204	10.0.2.15	18.192.151.104	MQTT	70	Subscribe Request (id=10) [house/#]
3303	20.224858918	10.0.2.15	18.192.151.104	MQTT	75	Subscribe Request (id=9) [university/#]
375	5.113041615	10.0.2.15	18.192.151.104	MQTT	80	Subscribe Request (id=3) [university/+/#]
2442	13.175483992	10.0.2.15	18.192.151.104	MQTT	87	Subscribe Request (id=5) [university/room0/room1/#]
3693	26.268559277	10.0.2.15	18.192.151.104	MQTT	91	Subscribe Request (id=13) [factory/department3/floor0/#]
3362	21.206357493	10.0.2.15	18.192.151.104	MQTT	94	Subscribe Request (id=15) [university/building2/section0/#]

Figure 1.2: CQ3: Filtered MQTT Subscriptions with Multi-Level Wildcards (#)

mqtt.msgtype == 1 and mqtt.conflog.willflag == 1 and mqtt.willtopic matches "^university(/.*)? \$"						
No.	Time	Source	Destination	Protocol	Length	Info
4	0.000117188	::1	::1	MQTT	176	Connect Command

Figure 1.3: CQ4: The structure of the MQTT CONNECT packet

In Wireshark, we can filter MQTT SUBSCRIBE messages by looking for packets with `mqtt.msgtype == 8`. Then, we filter the topics that contain the multi-level wildcard (#).

Filter Condition:

```
ip.dst == 18.192.151.104 && mqtt.msgtype == 8 && mqtt.topic
  ↳ contains "#"
```

Conclusion: Only count MQTT clients connecting to the public HiveMQ broker and subscribing using multilevel wildcard #. The different subscribing clients are filtered out (distinguishable based on ports), with a total of 4 clients subscribing to the public broker HiveMQ using multi-level wildcards, as shown in Figure 1.2.

1.4. CQ4: How many different MQTT clients specify a Last Will Message to be directed to a topic having as first level "university"?

We use the following filter to capture MQTT clients that meet the conditions:

```
mqtt.msgtype == 1 and mqtt.conflog.willflag == 1 and
  ↳ mqtt.willtopic matches "^university(/.*)? $"
```

In the MQTT protocol, the CONNECT packet is used to establish a connection between the client and MQTT broker and can optionally declare a Last Will Message (LWT). The structure of the MQTT CONNECT packet is as follows:

Conclusion:

After applying the Wireshark filter to capture packets meeting the CQ4 criteria, we will be able to identify all client connections with an enabled Last Will Message and a Will

Length(Byte)	Field Name	Purpose
Fixed Header		
...	...	...
1	mqtt.msgtype	4 bits: Must be 0001 (indicating CONNECT)
...	...	...
Connect Flags		
2	mqtt.conflag.willflag	1 = Enable Last Will Message, 0 = Disable LWT
...	...	...
41	mqtt.willtopic	Client's LWT topic, must start with "university/"
...	...	...

Table 1.4: CQ4: Key Field Analysis

mqtt.msgtype == 1 && mqtt.conflag.willflag == 1 && mqtt.willtopic matches "^([+#+])*\$"						
No.	Time	Source	Destination	Protocol	Length	Info
4	0.000117188	:::1	:::1	MQTT	176	Connect Command
196	2.116585177	10.0.2.15	5.196.78.28	MQTT	126	Connect Command
352	5.034840089	10.0.2.15	5.196.78.28	MQTT	123	Connect Command
557	7.043177949	10.0.2.15	5.196.78.28	MQTT	120	Connect Command

Figure 1.4: CQ5: Setting Will Message Mode

Topic starting with "university/". After filtering, the result is shown in Figure 1.3 and Table 1.4, where only 1 client satisfies the CQ4 condition.

1.5. CQ5: How many MQTT subscribers receive a last will message derived from a subscription without a wildcard?

The following filter command is used to identify the clients that have enabled the unexpected interrupt Will message mode. Based on Figure 1.4, a total of 4 clients were filtered out.

```
mqtt.msgtype == 1 && mqtt.conflag.willflag == 1 && mqtt.
  ↳ willtopic matches "^([+#+])*$"
```

After filtering for PUBLISH type messages, only those with the topic "university/department12/room1/temperature" contain PUB messages. Based on Figure 1.5, a total of 4 clients were identified.

```
mqtt.msgtype == 3 && (mqtt.topic == "university/department12/
  ↳ room1/temperature")
```

mqtt.msgtype == 3 && mqtt.topic == "university/department12/room1/temperature"						
No.	Time	Source	Destination	Protocol	Length	Info
6560	146.692096889	:::1	:::1	MQTT	162	Publish Message [university/department12/room1/temperature]
6562	146.692187516	:::1	:::1	MQTT	164	Publish Message (id=1) [university/department12/room1/temperature]
6566	146.692209983	:::1	:::1	MQTT	164	Publish Message (id=1) [university/department12/room1/temperature]
6564	146.692199911	:::1	:::1	MQTT	165	Publish Message (id=12) [university/department12/room1/temperature]

Figure 1.5: CQ5: Filtering PUBLISH Messages for Specific Topics

mqtt.msgtype == 8 && mqtt.topic == "university/department12/room1/temperature"						
No.	Time	Source	Destination	Protocol	Length	Info
121	1.083118347	:::1	:::1	MQTT	136	Subscribe Request (id=1) [university/department12/room1/temperature]
154	2.082593293	:::1	:::1	MQTT	136	Subscribe Request (id=1) [university/department12/room1/temperature]
304	4.097040463	:::1	:::1	MQTT	136	Subscribe Request (id=1) [university/department12/room1/temperature]

Figure 1.6: CQ5: Filtering Subscribers for Specific Topics

Lastly, the following filter command is used to identify the clients that subscribed to the "university/department12/room1/temperature" topic.

Concluding analysis: Based on the filtered results in Figure 1.6, a total of **3** clients subscribed to this topic. Their respective ports are: 53557, 39551, and 41789.

```
mqtt.msgtype == 8 && mqtt.conflag.willflag == 1 && mqtt.
  ↳ willtopic matches "[^+#+]*$"
```

Source Port: 53557

Destination Port: 1883

(a)

Source Port: 39551

Destination Port: 1883

(b)

Source Port: 41789

Destination Port: 1883

(c)

Figure 1.7: CQ5: Port Numbers of Will Message Receivers

1.6. CQ6: How many MQTT publish messages directed to the public broker mosquitto are sent with the retain option and use QoS "At most once"?

```
mqtt.msgtype == 3 and mqtt.retain == 1 && mqtt.qos == 0 && ip
  ↳ .dst_host == "5.196.78.28"
```

Field(Header Flags)	Purpose
<code>mqtt.msgtype == 3 (0011)</code>	Use the filter to select all PUBLISH messages.
<code>mqtt.qos == 0 (.... .00.)</code>	To select messages with QoS level 0, meaning "at most once" delivery.
<code>mqtt.retain == 1 (.... ...1)</code>	To filter messages where the retain flag is set to 1

Table 1.5: CQ6: Header Flags Field Analysis

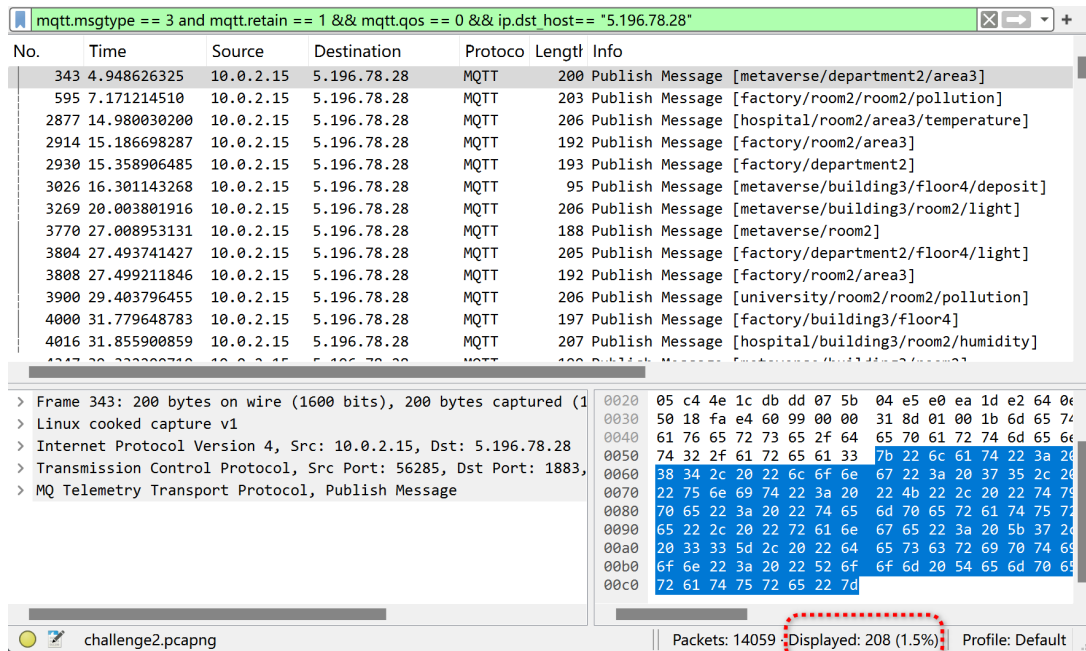


Figure 1.8: CQ6

After filtering with the above conditions, the final count of messages matching the criteria is **208**. This result is confirmed through the application of the filters and statistical analysis in Wireshark.

1.7. CQ7: How many MQTT-SN messages on port 1885 are sent by the clients to a broker in the local machine?

To extract the MQTT-SN messages sent by clients to the local broker, we need Wireshark filters..

```
udp && udp.dstport == 1885 && ip.dst == 127.0.0.1 && udp.
↳length > 2
```

udp && udp.dstport == 1885 && ip.dst == 127.0.0.1 && udp.length > 2						
No.	Time	Source	Destination	Protocol	Length	Info

Figure 1.9: CQ7

This filter:

Retains UDP packets (excludes TCP and other protocols).

Selects destination port 1885 (keeps only MQTT-SN messages).

Ensures messages are sent to the local broker (excludes messages sent to remote servers or between clients).

Keeps only valid MQTT-SN messages (removes invalid packets).

According to Wireshark, the number of packets matching CQ7 conditions is **0**.

